

INFO288: Buscador tipo Google SearchV16

Martin Alvarado, Eduardo Barrera, Isaias Cabrera, Osvaldo Casas-Cordero, Andrés Mardones

I. INTRODUCCIÓN

EN el presente informe se describe un sistema distribuido que pretende solucionar el problema de la ineficiente búsqueda de información dentro de una compañía ficticia. Para eso se presenta en primer lugar los detalles de la problemática, así como las hipotéticas especificaciones que una corporación podría entregar para responder preguntas orientadas a complementar la información con respecto al contexto de la empresa y entregar un producto más específico.

Luego, se detalla la solución propuesta, que está basada en un sistema distribuido. Un sistema distribuido es un conjunto de componentes independientes que trabajan de forma coordinada para actuar como un único sistema coherente frente al usuario. Este tipo de sistemas presentan múltiples ventajas relacionadas con el rendimiento, escalabilidad o el manejo de fallas y errores. Es por eso que el objetivo del proyecto es aprovechar estos beneficios con tal de desarrollar un buscador capaz de facilitar la localización de información entre una vasta cantidad de documentos. Así, el sistema será capaz de soportar un gran volumen de consultas y, además, podrá escalar de forma adecuada ante un crecimiento exponencial de documentos.

II. PROBLEMA DETECTADO (U OPORTUNIDAD DE INNOVACIÓN)

II-A. Descripción

En la actualidad, el volumen de datos generado por diversas fuentes crece de forma exponencial. El problema reside en la latencia y la ineficiencia de los métodos de búsqueda tradicionales basados en escaneo secuencial o consultas de texto parcial en bases de datos relacionales, los cuales no escalan ante millones de documentos.

Limitaciones detectadas:

- **Velocidad de respuesta:** A medida que aumenta la cantidad de datos, el tiempo de búsqueda se vuelve demasiado extenso para el usuario.
- **Dispersión de fuentes:** La dificultad de consolidar documentos de orígenes diferentes en una interfaz única.
- **Costo computacional:** Repetir cálculos complejos para búsquedas frecuentes agota los recursos del servidor sin necesidad.

La oportunidad radica en implementar una arquitectura de Sistema Distribuido que no solo centralice la información, sino que la procese de manera inteligente para ofrecer resultados instantáneos. La innovación se centra en tres pilares:

1. **Indexación mediante Índice Invertido:** En lugar de buscar una palabra dentro de cada documento, se crea un diccionario donde cada palabra (token) apunta a una lista de documentos que la contienen. Esto transforma una búsqueda de complejidad lineal en una búsqueda de acceso casi instantáneo.
2. **Estrategia de Caché Multinivel:** Implementar una capa de memoria volátil que almacene los resultados de las consultas más frecuentes. Esto permite que, para términos populares, el sistema ni siquiera tenga que consultar el índice en disco, reduciendo la latencia a milisegundos.
3. **Escalabilidad Horizontal:** Al ser un sistema distribuido, la carga de la indexación y la búsqueda se reparte entre varios nodos, permitiendo que el buscador crezca orgánicamente según la demanda.

II-B. Preguntas

Los supuestos de este sistema se presentan mediante preguntas a los clientes, junto con sus respectivas respuestas, estas son las siguientes:

¿Qué tipo de documentos van a ser buscados?

R. Documentos TXT en un principio.

¿Cuál es el volumen de consultas esperado?

R. Se esperan al menos 100 consultas por segundo.

¿Cuál es el origen de los documentos? ¿De dónde se ingresan al sistema?

R. El origen de los documentos será un repositorio de Google Drive donde se guardan los textos.

¿Cuántos servidores se tienen y cuales son sus características?

R. Se cuentan con 5 computadores especificados en el apartado *IV Modelo físico tabla II*

¿Con cuántos documentos se cuenta al inicio y cuál es la tasa de crecimiento esperada?

R. Inicialmente manejamos unos 100. Esperamos crecer un 30 % anual a medida que integremos nuevas fuentes.

Con respecto a actualizaciones de documentos ¿Se deben reflejar los cambios en tiempo real o es aceptable un retraso?

R. Máximo 5 minutos de retraso para reflejar los cambios.

¿Cuál es el nivel de disponibilidad requerido?

R. Si bien no es un sistema crítico, queremos que el sistema esté disponible la mayoría del tiempo para así facilitar el uso de nuestros servicios.

¿Necesitan herramientas de monitoreo y alertas sobre el estado del sistema?

R. Por el momento, no consideramos necesario implementar herramientas de monitoreo externas. Dado que estamos en una etapa inicial, preferimos simplificar la infraestructura para mantener los costos bajos y el despliegue lo más directo posible.

¿Qué aspectos de accesibilidad son requeridos para la interfaz?

R. Debe incluir el modo nocturno y una opción para activar los colores de alto contraste.

¿Qué aspectos debe tener en cuenta la interfaz de usuario?

R. Necesitamos una interfaz simple y rápida, algo similar a un buscador de google.

III. SOLUCIÓN PROPUESTA Y DISEÑO

III-A. Descripción de diagrama de componentes

Interfaz Gráfica: será desarrollada en Astro en su versión 6.13 (framework basado en JavaScript) y permitirá al usuario ingresar las búsquedas para luego mostrar los textos resultantes ordenados según los cálculos realizados por el backend.

Query Router: la principal función de este componente será recibir las llamadas a la API efectuadas por el frontend (usando FASTAPI v0.135, un framework basado en python).

Caché: en este componente se almacenan las últimas búsquedas y van saliendo mediante un puntaje asignado que depende de cuántas búsquedas han pasado desde que fue consultado por última vez. (Redis v7.4.0).

DB distribuida índices invertidos: para este componente se usará Apache Cassandra v5.0, la cual es NOSQL y será distribuida en diferentes nodos para almacenar los datos relacionados a cada palabra.

Merger + Ranker: este componente se encarga de calcular los puntajes de los textos resultantes de la búsqueda, interactuando directamente con las bases de datos. La comunicación con el Query Router será mediante gRPC v1.78.1. (framework en python), que utiliza un protocolo más ligero construido sobre TCP, que otorgará agilidad en la comunicación interna.

DB documentos: esta base de datos almacenará la información relativa a los documentos y será clave para mostrar sus metadatos al usuario.

Parser + Indexer: este componente desarrollará 2 funciones fundamentales. La primera será procesar los documentos para obtener los índices invertidos. Esto incluirá calcular TF-IDF de las palabras, eliminar stopwords o stemming. Y por otro lado, será el encargado de extraer los metadatos de los documentos del google drive mediante la API adaptada a Python.

Repositorio documentos: aquí se encuentran todos los documentos del sistema, se optó por utilizar Google Drive para este fin.

Cabe destacar que todo el entorno de trabajo estará basado en el lenguaje Python en la versión 3.11. Todos los frameworks son de uso libre con tal de reducir al máximo los costos para el cliente y fueron elegidos debido a su cómoda adaptación con el lenguaje a utilizar.

Los únicos puertos que se establecerán como públicos son los del frontend (3000) y de la API de entrada que contiene el Query Router (8000). Los demás no conviene exponer al usuario porque solo sirven para la comunicación interna del sistema.

III-B. Diagrama de componentes

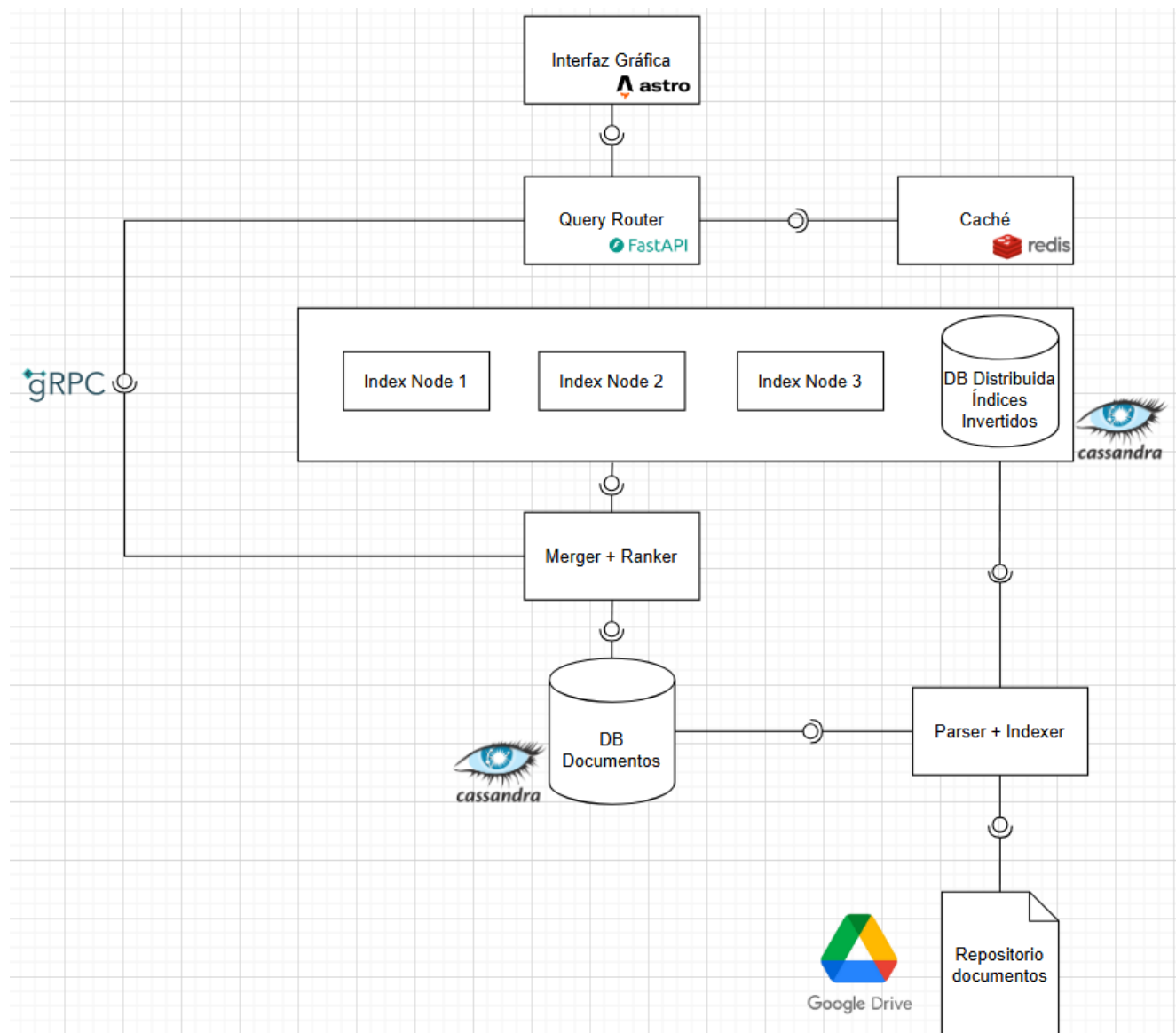


Figura 1. Diagrama de componentes del sistema de búsqueda distribuido.

III-C. Modelos fundamentales

III-C1. Interacción: A continuación se explica cómo se comportará el sistema con respecto a los casos de usos más importantes.

- **Caso Búsqueda:** El sistema recibe en un primer momento la consulta de parte del usuario mediante una cadena de texto. El componente encargado de procesar esta solicitud es el *Query Router*, que será efectuada mediante una llamada a la API especificada en el *backend* (asíncrona). Este componente procederá a buscar en la caché mediante el protocolo RESP, debido a la tecnología REDIS de la caché. En el caso de haber un *caché hit*, se devuelve la lista de resultados que fue previamente calculada y almacenada en caché.

Si no existe coincidencia para la búsqueda ingresada en la memoria caché del sistema, se continúa con la búsqueda en los *Index Nodes*, efectuada por el *Merger + Ranker*, el cuál hará una consulta a la base de datos distribuida para obtener las estadísticas de las palabras incluidas en la búsqueda. Luego, procederá a calcular los puntajes para finalmente devolver un *ranking* con los textos más afines a los términos buscados. El *Merger + Ranker* consulta los metadatos de los textos en la base de datos de documentos para enviar toda esta información al *Query Router*, quien se encargará de mostrar los resultados de la búsqueda al usuario. Cabe destacar que esta comunicación entre *Merger + Ranker* y el *Query Router* será ejecutada mediante el *framework gRPC*, que utiliza un protocolo de comunicación más rápido que una API REST.

- **Caso Nuevo Archivo:** Cuando un nuevo archivo es escrito en los *Doc Stores*, un componente de detección de cambios registra el evento y publica una solicitud de procesamiento en una cola de trabajo. Un *worker* consume esta solicitud, extrae el contenido textual del documento y lo somete al *pipeline* de preprocesamiento NLP, que realiza la tokenización, normalización y *stemming* del texto. Con el texto procesado, se actualizan los *Index Nodes* con las nuevas entradas del documento. Finalmente, se ejecuta una invalidación selectiva de caché para garantizar que las búsquedas afectadas reflejen el nuevo contenido.
- **Caso Eliminación de un archivo:** Para eliminar un archivo, el sistema borra el documento de los *Doc Stores*, pero mantiene su identificador en los *index Notes* para evitar la sobrecarga de editar los *Index Nodes* en tiempo real. Para solucionar esto, se registra el ID del documento en una lista de exclusión que el *Merger + Ranker* consulta en cada búsqueda, filtrando los resultados antes de entregarlos al usuario. Simultáneamente, se ejecuta una invalidación de la caché, eliminando de la memoria volátil cualquier consulta previa que contenga el ID del archivo borrado para evitar que el *Query Router* entregue datos obsoletos. Este proceso garantiza que el usuario no vea el archivo inmediatamente, aunque este se encuentre todavía en los *Index Nodes*. Para finalizar este proceso, el rastro del documento se elimina por completo de los *Index Nodes* solo cuando se ejecuta el proceso de reindexado, el cual reconstruye los índices desde 0 basándose en el estado actual de los *Doc Stores*.
- **Caso Modificación de un archivo:** El sistema opera bajo una lógica de “eliminar y crear”, primero se ejecuta el borrado lógico agregando el ID del documento a la lista de exclusión del *Merger + Ranker*, invalidando al mismo tiempo la caché para evitar resultados obsoletos en esta. Inmediatamente después, el documento con los cambios se procesa como un archivo nuevo (Caso: Nuevo Archivo). Este flujo permite que el usuario vea la información nueva de inmediato sin que el sistema sufra una sobrecarga de procesamiento.

III-C2. Manejo de fallos: Dado que el sistema opera sobre 5 nodos con hardware distinto, la estrategia se centra en la tolerancia a fallas y la recuperación para asegurar que el buscador siga operativo incluso si uno de los computadores sufre una caída.

Tabla I
MATRIZ DE FALLOS Y ESTRATEGIAS DE MITIGACIÓN

Tipo de fallo	Componente	Estrategia de Mitigación
Caída de Nodo	Index Nodes	El índice invertido se divide en fragmentos. Cada fragmento tendrá al menos una réplica en un nodo distinto. Si el Nodo 2 cae, el Nodo 4 que tiene la réplica asume las consultas.
Falla en caché	Redis (Caché)	Si el servicio de caché no responde, el Query Router redirige las consultas directamente a los Index Nodes. El sistema será más lento, pero seguirá entregando resultados.
Corrupción	Doc Store	Al cargar archivos, se genera un hash (MD5/SHA). El sistema verifica el hash antes de procesar el archivo para asegurar que no se haya corrompido durante la transferencia.
Sobrecarga	Backend / API	Ante un pico superior a las 100 consultas por segundo, el sistema implementará una cola de espera para evitar el colapso de la RAM en los nodos más débiles.

III-C3. Diseño de Seguridad: El diseño de seguridad del sistema distribuido se enfoca en proteger los canales, procesos y objetos, mitigando vulnerabilidades tanto externas como internas. A nivel perimetral, la interacción con el usuario se realiza mediante HTTPS para establecer canales seguros que aseguran la privacidad e integridad de las consultas. Además, para evitar la saturación del servidor producto de una sobrecarga de solicitudes y sostener el rendimiento esperado, el nodo de entrada aplica limitación de tasa (*Rate Limiting*), previniendo amenazas de negación de servicios (*DoS*). Como medida preventiva secundaria frente a la potencial inyección de código malicioso a los servidores, los documentos ingresan a la plataforma habiendo sido previamente saneados por un proceso externo.

Para resguardar la infraestructura interna, la comunicación entre los componentes del buscador utiliza el protocolo *gRPC* respaldado por autenticación mutua (*mTLS*). Esto impide que un proceso atacante ocupe una identidad falsa para acceder a la memoria caché o los nodos de indexación. Mediante el uso de técnicas de encriptación y secreto compartido, los servidores validan rigurosamente la identidad de cada nodo antes de procesar cualquier solicitud interna, garantizando un entorno seguro y de confianza estricta.

IV. MODELO FÍSICO

Tabla II
CARACTERÍSTICAS DE LA INFRAESTRUCTURA DE CÓMPUTO

Comp.	CPU	GPU	RAM	Almacenamiento
1	Ryzen 5 4600H	GTX 1650Ti	8 GB	512 GB SSD
2	i5-11400H	GTX 1650	16 GB	512 GB SSD
3	i5-12500H	Iris® Xe Graphics	16 GB	256,1 GB SSD
4	i5-11400H	GTX 1650	16 GB	256 GB SSD
5	i5-14400F	RX 6700 XT	16 GB	931GB HDD / 832GB SSD

V. MODELO DE BASE DE DATOS

El sistema utiliza una arquitectura de almacenamiento híbrida basada en *Apache Cassandra*, aprovechando su capacidad para manejar datos distribuidos y escalabilidad horizontal. A continuación, se detallan los esquemas de las tablas que componen el *Inverted Index database* y el *Doc Stores database*.

V-A. *Inverted Index Database*

Esta base de datos es el núcleo del motor de búsqueda, permitiendo la localización rápida de términos a través de un índice invertido y gestionando la exclusión de documentos eliminados.

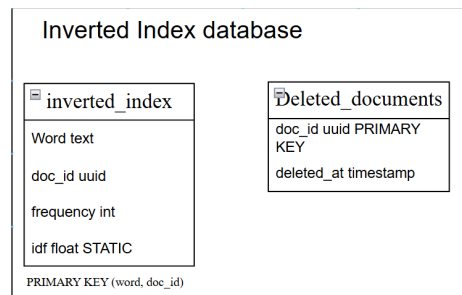


Figura 2. Esquema lógico de la base de datos de índice invertido.

Tabla III
ESPECIFICACIÓN DE LA TABLA *inverted_index*

Columna	Tipo (CQL)	Rol Cassandra	Descripción
word	text	Partition Key	La palabra o término indexado (token). Determina en qué nodo se guardan los datos.
doc_id	uuid	Clustering Column	Identificador único del documento. Ordena los resultados dentro de la partición.
frequency	int	Column	<i>Term Frequency</i> (TF): Cantidad de veces que la palabra aparece en el documento.
idf	float	Static Column	<i>Inverse Document Frequency</i> : Peso global de la palabra. Se almacena una sola vez por <i>word</i> .

Tabla IV
ESPECIFICACIÓN DE LA TABLA *deleted_documents*

Columna	Tipo (CQL)	Rol Cassandra	Descripción
doc_id	uuid	Primary Key	El identificador único del documento que fue eliminado.
deleted_at	timestamp	Column	Fecha y hora exacta en la que se marcó el borrado para procesos de limpieza.

V-B. Doc Stores Database

Esta base de datos almacena los metadatos necesarios para la recuperación y visualización de los archivos originales.

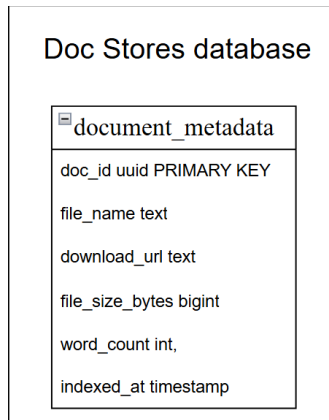


Figura 3. Esquema de metadatos en Doc Stores database.

Tabla V
ESPECIFICACIÓN DE LA TABLA *document_metadata*

Columna	Tipo (CQL)	Rol	Descripción
doc_id	uuid	Primary Key	Identificador único coincidente con el ID del índice invertido.
file_name	text	Regular	El nombre original del archivo (ej: "tesis_final.pdf").
download_url	text	Regular	Enlace directo para la descarga del archivo.
file_size_bytes	bigint	Regular	Tamaño del archivo en bytes.
word_count	int	Regular	Cantidad total de palabras procesadas en el documento.
indexed_at	timestamp	Regular	Fecha en la que el documento fue ingresado al sistema.